Git DOs & DON'Ts

DO use branches

The most important feature of git is its branching model. These branches are more like a new copy of your code's current state.

Before you start working on a change create a new branch for it:

- branch for a new feature
- branch for bug fixes
- branch for some issue

Benefits of branches:

- Easier to see which feature/bug/issue is being worked on
- Allows multiple people to work on multiple things at once

DO use branch naming conventions

Branch naming conventions make it easier to see what feature/bug/issue is being worked on branches.

All collaborators should agree on group words, separators and order of naming to use when naming branches. Usually the order of naming follows something along the lines of `<issue type> → <concise issue description> → <issue ticket number>`

Example (Yes, slashes can be used in branch names 😉):

- feature/offline-help/ROG-1234
- bug/crash-on-unsecure/ADM-1337
- issue/acc-py-deployment/LUL-0451

Example 2:

- f-offline_help-ROG_1234
- b-crash_on_unsecure-ADM_1337

- i-acc_py_deployment-LUL_0451

For some groups, 100% consistent naming is not as important, but it is important to **use atleast some kind of convention**.

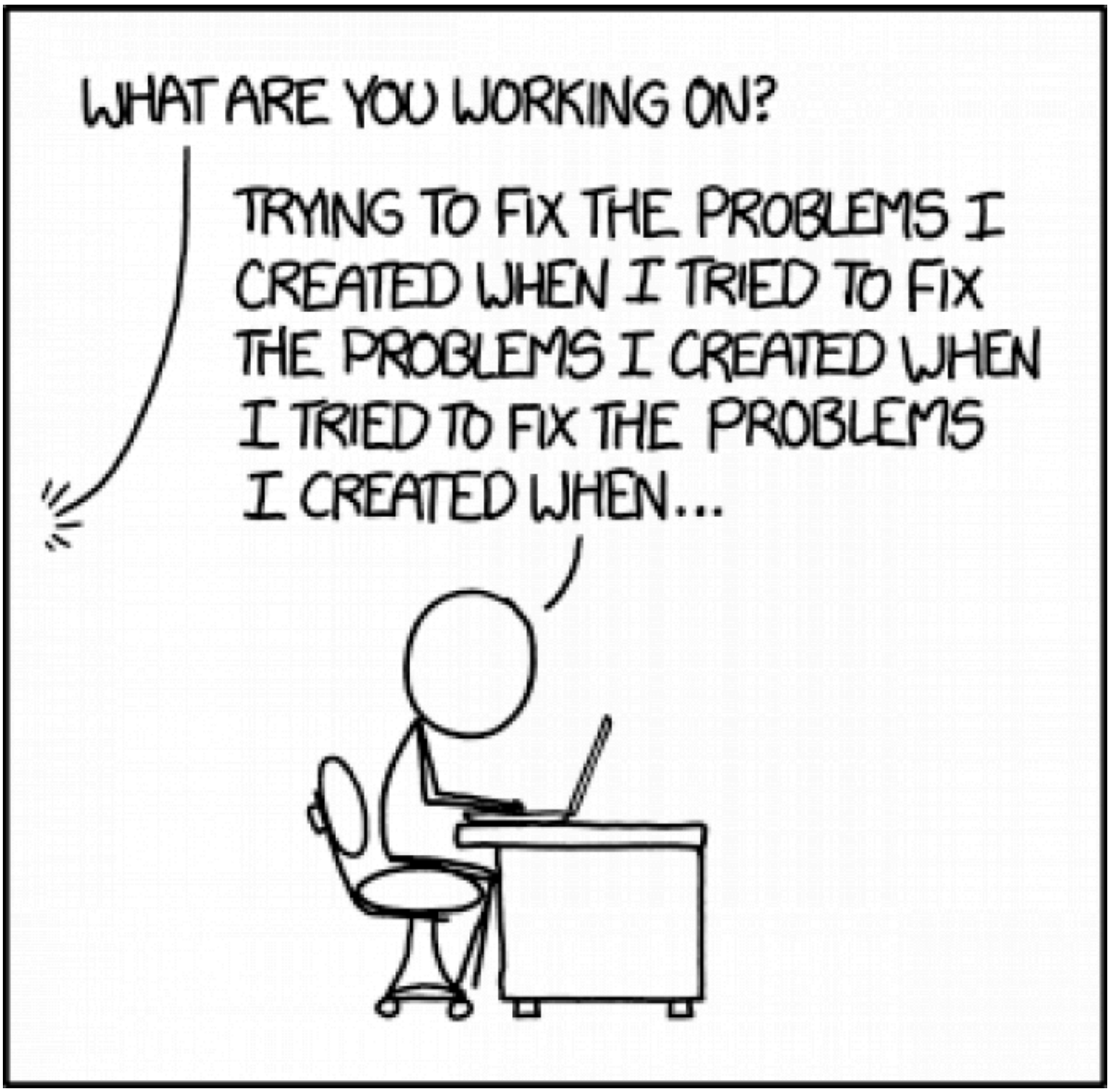
DO atomic commits

Atomic commit is the smallest, most important improvement you can make in your source code.

Atomic commit = one commit for one change.

If you have made a change that **works** then commit it. Having periodic checkpoints like this means that you can understand how you broke something.

The more changes you accumulate the harder it gets to track down the root cause of a breakage:



How to do atomic commits:

- By working on one thing at a time
- Keep your changes small

Benefits of atomic commits:

- Easier to read
- Easier to track
- Easier to review

DON'T commit broken code

Atomic commits are great, but they **should be commited only when the changes work**. Remember, you are very likely working in collaboration with other people. Your collaborators might want or need to branch off of your commits (and maybe not necessarily the latest commit at that) and they will soon be pulling their hair out if they can not figure out why their changes are broken, when in reality it was rigged from the start.

DO squash commits

Squashing commits melts all commits into the first one.

This is especially important if you've changed a lot of things back and forth or done some nonesensical commits for testing. Anyone working with GitLab CI should understand the pain:

Doing atomic commits is good, but having a readable git log is also important. So when merging your changes from your appropriately named branch into the main branch, **consider squashing your commits.**

For some groups squashing is something to be avoided to keep a very detailed log and preserve the benefits for atomic commits on the main branch. For other, especially larger groups, squashing commits is the only way to keep the git log readable and manageable. Having the freedom to do as many, even nonsensical commits, on your own branch is a

necessary freedom, squashing on merge allows that freedom while preserving readability on the main branch.

DO use a commit template/style

Continuing with making the git log more readable, commit message templates are a great way to improve readability. There are many commit styles, so one has to be agreed upon with the team.

Most commit styles compose of a tag and some sort of description.

Example of a simple commit message template:

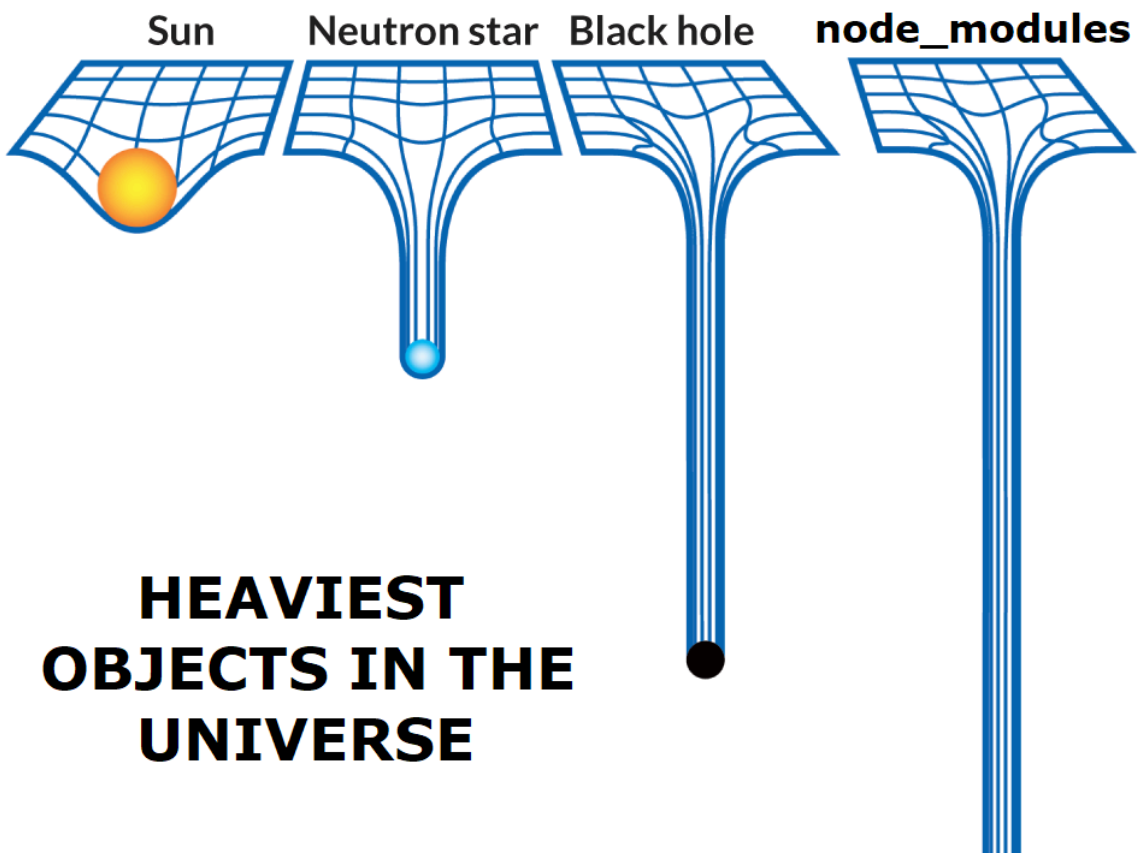
Text Only

```
<tag>: <subject>
<body>
```

Unified commit style allows the usage of git log in automated tasks such as automatically generating a changelog, programatically figuring out which version to set the tag as among others.

DON'T commit generated files and dependencies, DO use .gitignore

Many projects have dependencies and routines that generate some kind of output, cache, local config files or otherwise some generated files. Save repository size and use .gitignore file to specify files that should not be tracked and pushed to the repository. Dependency management should be done with a build/dependency tool tailor made for your project's software stack. Not to mention, dependencies will often prove to take the most space relative to the source code:



Benefits of using .gitignore:

- Keeps the repository organized
- Keeps the repository size small

DON'T commit secrets

This technically goes under the previous chapter, but is important enought to warrant a separate one.

Config files and maybe even source code might need to contain secrets like passwords, API keys. But **NEVER** keep them there in plaintext even if your repository is private. There are many tools available for this, including:

- GitLab Variables
- Your teams key/value store or secrets engine

So use them, because remember - previous commits are etched into the git history so even if your commit over the commit that had secrets in it, the secrets will still always be readable from the git history.

DON'T rewrite git history

This is of course an exception if you've pushed some secrets to your repository. However in your usual workflows you should ideally consider pushed commits etched in diamond for all eternity. If you later find out that you messed up, make new commits that fix the problems (possibly by revert, possibly by patching, etc).

Yes, of course git allows you to rewrite public history, but it is problematic for everyone else that is working with the same repository and thus it is just not best practice to do so.

If you need to rewrite history, then you should discuss it in depth with your collaborators.

DO use code review

Use merge/pull requests where possible. Try not to merge into the main branch at your leisure as it may mess up others work. This goes double for repositories that use automated continous integration and deployment and triple for repositories that are used by a lot of people and quadtruple for repositories that are critical.

Create a request at a team members name that is familiar with the repo so that they can provide a fresh perspective on the changes before mergin to main branch.

DO use pre-commit

Git hook scripts are useful for identifying simple issues before submission to code review.

Easiest example: it will automatically format codes when commiting.

DO use CI/CD

CI/CD is a tool for software development using the continuous methodologies:

- Continuous Integration (CI)
- Continuous Delivery (CD)

- Continuous Deployment (CD)

CI/CD is used to automatically build, test, deploy, and monitor your applications to catch bugs and errors early in the development cycle.

Manually deploying applications is time consuming and error-prone. With CI/CD you can automate that to avoid errors and save time in the process.

To use GitLab CI/CD, create a file called `.gitlab-ci.yml` and in there specify in YML the steps that the CI/CD should run.

CI/CD steps are called pipelines that consist of stages that consist of tasks.

So for example a "deployment" pipeline would consist of stages "copy configuration files", "update to latest version" and "start application". And each stage would have YML tasks that are very similar to Ansible in syntax.

Each stage is executed by a docker container called Executor that is spawned by the Runner. After a stage is finished, the docker container for that Executor is destroyed.

With GitLab CI/CD you have many powerful features for describing which conditions must be met for pipeline/stages/tasks to run. They may run in parallel or sequentially, on schedule, on commit or when a variable is set manually. The possibilities are almost endless and are all covered in the official Documentation: https://docs.gitlab.com/ee/ci/